Conditions of Parallelism

Bernstein's Conditions

- Bernstein's revealed a set of conditions which must exist if two processes can execute in parallel.
- Notation:
  - Let P_1 and P_2 be two processes .
  - I_i is the set of all input variables for a process P_i .
  - O_i is the set of all output variables for a process P_i .
- If P_1 and P_2 can execute in parallel (which is written as $P_1 \parallel P_2$), then:

$$I_1 \cap O_2 = \emptyset$$

$$I_2 \cap O_1 = \emptyset$$

$$O_1 \cap O_2 = \emptyset$$

Bernstein's Conditions

- In terms of data dependencies, Bernstein's conditions imply that two processes can execute in parallel if they are flow-independent, anti-independent, and output-independent.
- The parallelism relation $\|$ is commutative ($P_i \| P_j$ implies $P_j \| P_i$), but not transitive ($P_i \| P_j$ and $P_j \| P_k$ does not imply $P_i \| P_k$) .
- Intersection of the input sets is allowed.

- **Example:** Consider the following statements:

P1: a = x + y

P2: b = x + z

Do they satisfy Bernstein's conditions.

Solution:

$I_1 = (x, y)$; $O_1 = (a)$

$I_2 = (x, z)$; $O_2 = (b)$

Therefore, $I_1 \cap O_2 = \emptyset$

$I_2 \cap O_1 = \emptyset$

$O_1 \cap O_2 = \emptyset$

Bernstein's conditions satisfied - these two statements can be executed in parallel.

- **Example:** Consider the following statements:

P1: a = x + y

P2: b = a + b

Do they satisfy Bernstein's conditions.

Solution:

$I_1 = (x, y)$; $O_1 = (a)$

$I_2 = (a, b)$; $O_2 = (b)$

Therefore, $I_1 \cap O_2 = \emptyset$

$I_2 \cap O_1 \neq \emptyset$

$O_1 \cap O_2 = \emptyset$

Bernstein's conditions not satisfied - these two statements cannot be executed in parallel.

- **Example:** Detect the parallelism in the following five statements:

P1: C = D * E

P2: M = G + C

P3: A = B + C

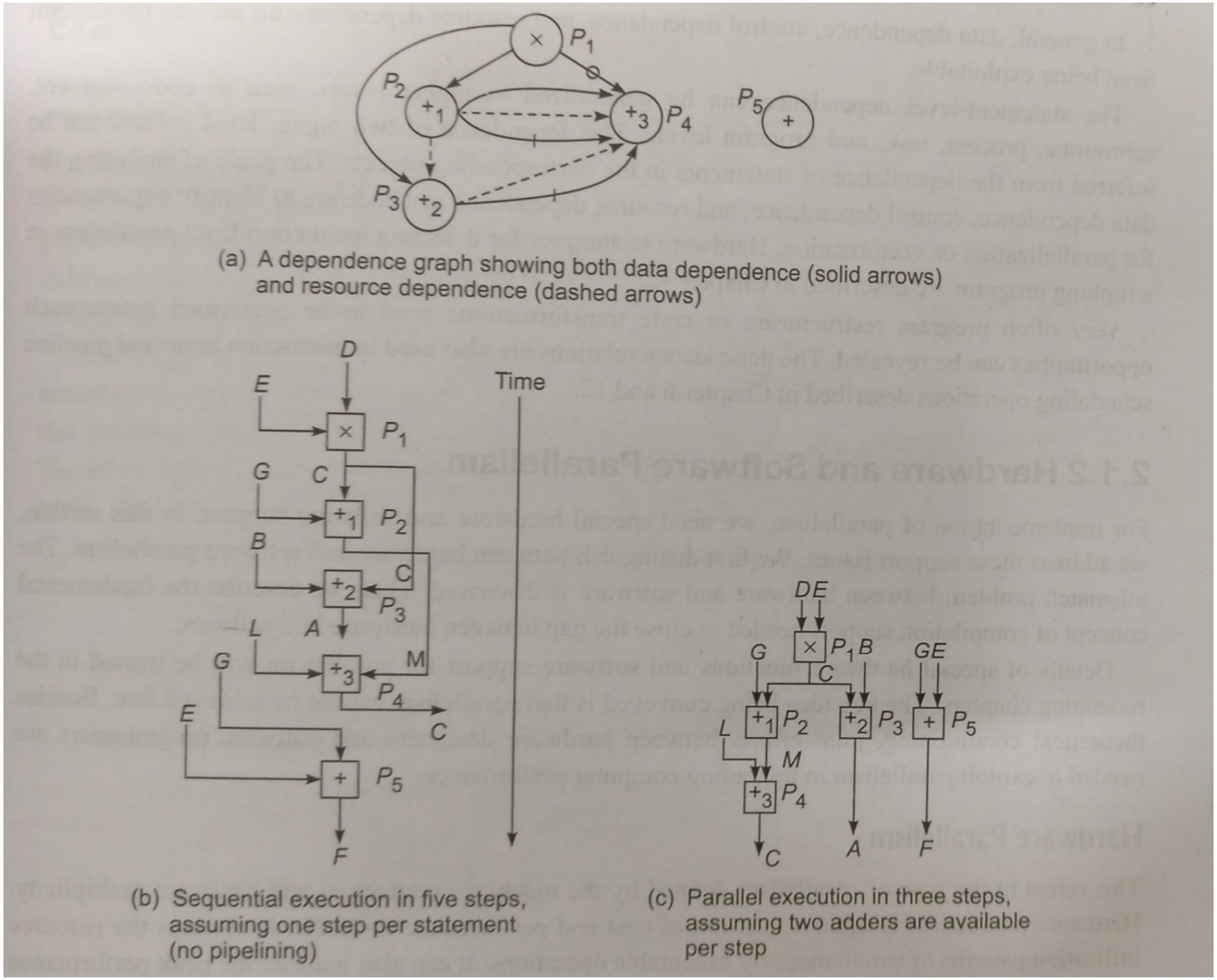
P4: C = L + M

P5: F = G / E

Assume that each statement requires one cycle to execute. No pipelining is considered here.

Show the dependence graph (both data dependence & resource dependence), sequential execution flow and parallel execution flow (assuming two adders are available simultaneously).

Using Bernstein's Conditions after checking statement pairs: P1 || P 5 , P2 || P 3 , P2 || P 5 , P3 || P 5 , P4 || P 5



Hardware and Software Parallelism

- **Example:** Detect the parallelism in the following five statements:

P1: C = D * E

P2: M = G + C

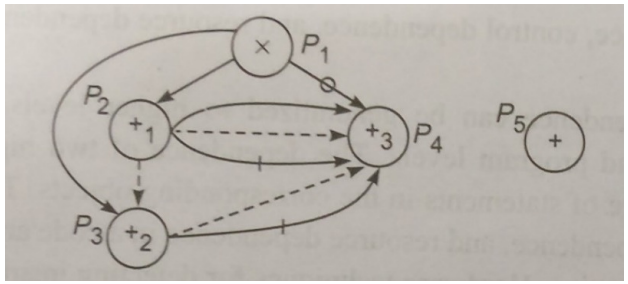
P3: A = B + C

P4: C = L + M

P5: F = G / E

Assume that each statement requires one cycle to execute. No pipelining is considered here.

Show the dependence graph, sequential execution flow and parallel execution flow (assuming one adder is available ; two adders are available; and three adders are available simultaneously).



Using Bernstein's Conditions after checking statement pairs: P1 || P 5 , P2 || P 3 , P2 || P 5 , P3 || P 5 , P4 || P 5

- **With one adder:**

Step 1: P1, P5

Step 2: P2

Step 3: P3

Step 4: P4

- **With two adders:**

Step 1: P1, P5

Step 2: P2, P3

Step 3: P4

- **With three adders:**

Step 1: P1, P5

Step 2: P2, P3

Step 3: P4

Hardware And Software Parallelism

- For implementation of parallelism, special hardware and software support is needed.
  - Compiler support is also needed to close to gap between hardware and software.
  - Redesigning hardware for exploiting parallelism.

Hardware Parallelism

- Hardware parallelism is defined by machine architecture and hardware multiplicity.
- It displays the resources utilization patterns of simultaneously executable operations.
- It can be characterized by the number of instructions that can be issued per machine cycle.
  - If a processor issues k instructions per machine cycle, it is called a k-issue processor.
  - Conventional processors are one-issue machines. Intel i960CA is a three-issue processor (arithmetic, memory access, branch). IBM RS- 6000 is a four-issue processor (arithmetic, floating- point, memory access, branch).

Software Parallelism

- Software parallelism is defined by the control and data dependence of programs, and is revealed in the program's flow graph.
- It is a function of algorithm, programming style, and compiler optimization. The program flow graph displays the patterns of simultaneously executable operations.

- **Example:** Consider the following program:

S1: Load R1, 100

S2: Load R2, 5

S3: Load R3, 20

S4: Load R4, 50

S5: R5 <- R1 * R2

S6: R6 <- R3 * R4

S7: R7 <- R5 + R6

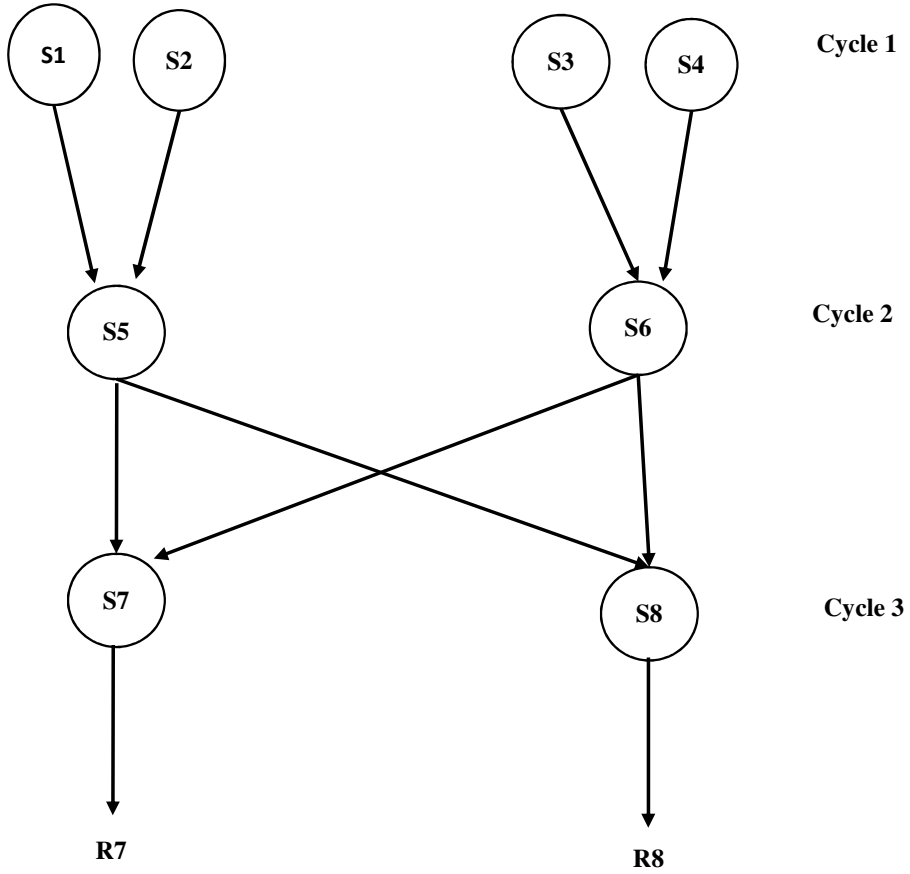
S8: R8 <- R5 – R6

Assume that each statement requires one cycle to execute.

Show program graph for both software and hardware parallelism.

Calculate average software parallelism and average hardware parallelism (considering two-issue processor which can issue one memory access and one arithmetic operation simultaneously).

Software Parallelism

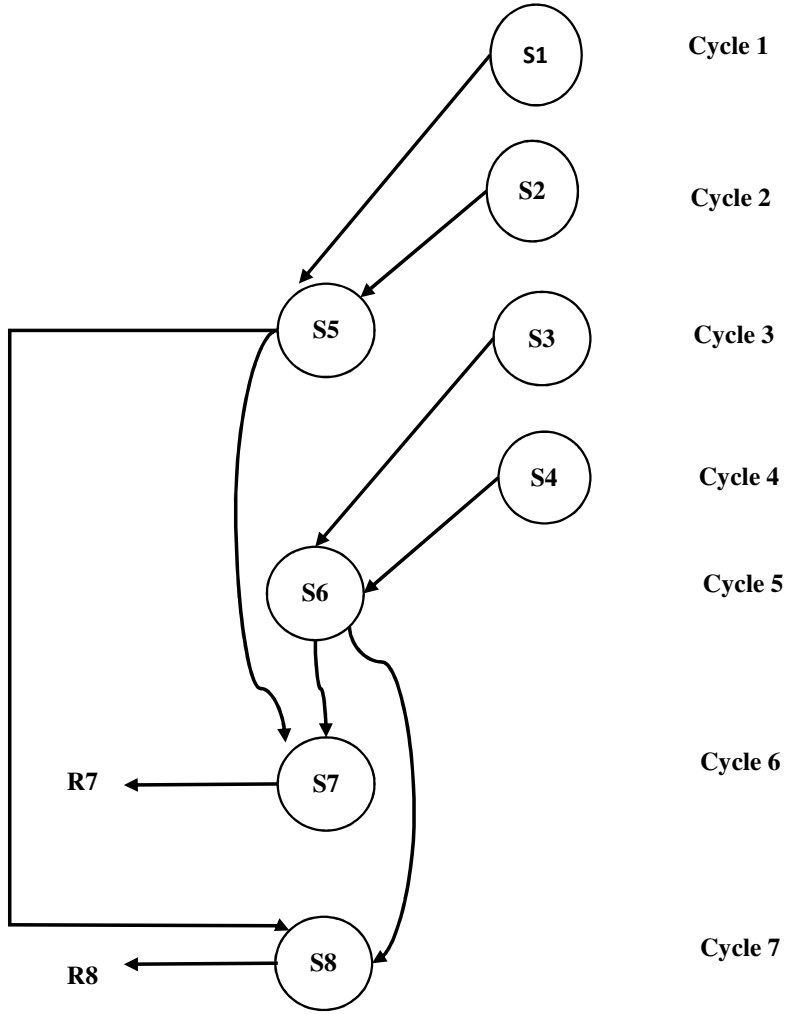


Average Software Parallelism

= 8/3

= 2.67 instructions per cycle

Hardware Parallelism

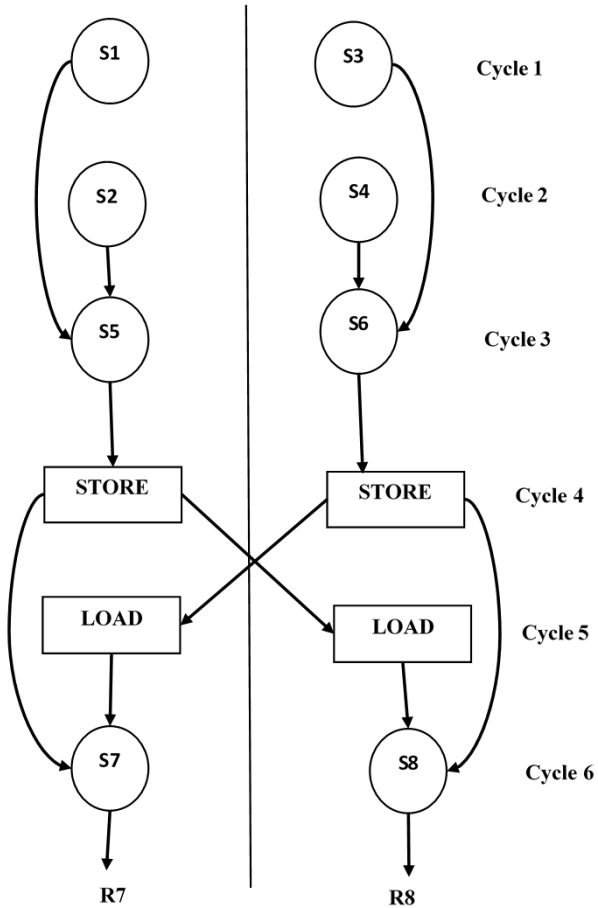


Average Hardware Parallelism

= 8/7

= 1.14 instructions per cycle

Considering a dual processor system, where single-issue processors are used:



Average Hardware Parallelism

= 12/6 instructions per cycle

(4 instructions for inter processor communication)

- **Example:** Detect the parallelism in the following five statements:

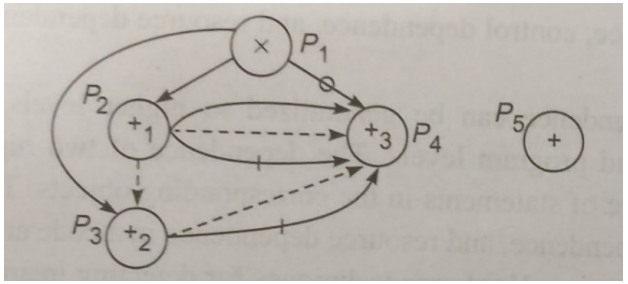
P1: C = D * E

P2: M = G + C

P3: A = B + C

P4: C = L + M

P5: F = G / E



Using Bernstein's Conditions after checking statement pairs: P1 || P 5 , P2 || P 3 , P2 || P 5 , P3 || P 5 , P4 || P 5

Assume that each statement requires one cycle to execute.
a) Calculate the average software parallelism.
b) Calculate the average hardware parallelism for a two-issue processor (can execute one data transfer and one arithmetic operation simultaneously).

How will your answer change if we assume 2 cycles for add unit, 3 cycles multiplication unit and 3 cycles for division unit.